When Should Web-RAG Refresh? Adapting Refresh Policies Across Drift Regimes

First Author[1*], Second Author[1,2] and Third Author[1]

[1*]Department of Computer Science, University Name, Street Address, City, 100000, State, Country.
[2]Research Lab, Institute Name, Street Address, City, 100000, State, Country.

*Corresponding author(s). E-mail(s): iauthor@gmail.com;
Contributing authors: iiauthor@gmail.com; iiiauthor@gmail.com;

Code: https://anonymous.4open.science/r/wise-rag-refresh-4606

etrieval-augmented generation, Web information retrieval, Temporal query intent, Cache refresh, Domain adaptation, Cost-aware systems

1 Introduction

Web-RAG systems answer questions by retrieving web pages and conditioning a large language model (LLM) on the evidence. To keep latency and cost down, deployed systems cache retrieved pages; but the web is the system's source of truth and it does not sit still. Prices and leadership change, and every cached page ages against its live counterpart. The system therefore faces a per-query decision: answer from the cache, or pay for a live refresh of the evidence?

This decision has a price on both sides, and which side dominates is an empirical property of the corpus we call its *drift regime* (Fig. 1). When cached and current pages barely differ (low drift), refreshing buys nothing and can actively hurt: the fetched text perturbs the evidence the generator conditions on. When they differ substantively (high drift), skipping the refresh means answering from outdated evidence. A single fixed policy cannot be right in both regimes: as we show, blind Always-Refresh is the *worst* system on a low-drift corpus (F1 0.173 vs 0.192 for never refreshing) yet the *best* on a high-drift one (F1 0.705 vs 0.191).

Existing work stops short of this decision. Temporal-IR and freshness benchmarks Vu et al. (2024b) measure *how much* the web changes or *when* stale context

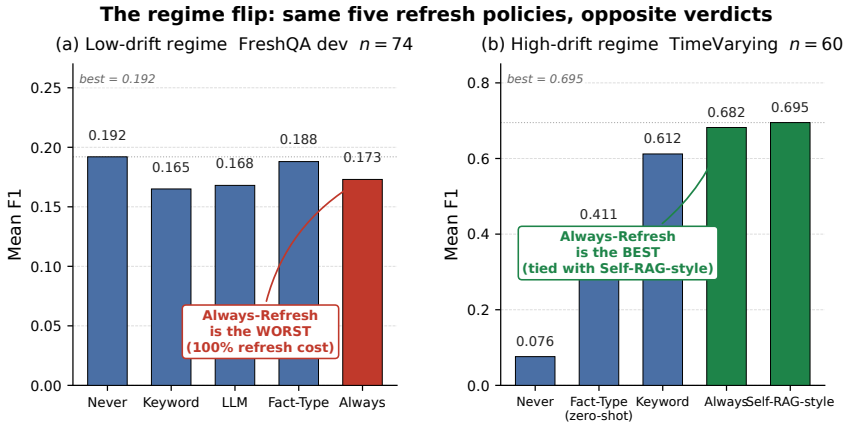


Fig. 1 The regime flip. The same five refresh policies (Never, Always, Keyword, LLM, Fact-Type Router) on two corpora: in the low-drift regime (left, FreshQA dev, n=74) blind Always-Refresh is the *worst* system (F1 0.173 vs 0.192 for Never) because refreshed evidence is a perturbation; in the high-drift regime (right, TimeVarying, n=60) it is the *best* (F1 0.705 vs 0.191) because stale evidence is simply wrong. Axes show mean token-level F1 against refresh rate. No fixed policy wins in both regimes, motivating a regime test (which family to ship) plus an adaptable router (the high-drift family's best member).

hurts, but do not learn a refresh policy; adaptive-retrieval methods Jiang et al. (2023); Asai et al. (2024) decide whether to retrieve against a *static* index and never condition on cache age; web-cache refresh heuristics optimize cache freshness and ignore downstream answer quality. Cost-aware refresh of RAG evidence, meaning the per-query decision of whether a live fetch is worth it and how that decision changes across drift regimes, remains unstudied at the system level.

We answer three questions on FreshQA (low drift, real Wayback snapshots Vu et al. (2024b)) and a high-drift corpus we construct (n=60), separated by Cohen's d=2.15 on per-query drift: (RQ1) how much staleness hurts answer quality; (RQ2) does the best refresh policy depend on the drift regime; (RQ3) can a learned refresh router transfer across regimes and be repaired with few labels. Our method has three components, a **Freshness Router** that scores queries with a refresh probability, a **Refresh-aware Pipeline** that isolates evidence freshness from retrieval, and a **Few-shot Adaptation** step. Two recipes emerge from this stack:

- **AdaRefresh** (transfer-and-adapt): a proxy-trained router collapses zero-shot across regimes (F1 0.422); 5 epochs of adaptation on $k \geq 4$ target queries recovers it to 0.685, above every rule-based policy. The from-scratch control at the same budget plateaus at 0.19: the gain comes from the proxy prior.
- **RegimeTest** (regime classification): averaging a cheap drift predictor over ∼20 held-out queries separates the two regimes with no overlap and selects the policy family within 0.02 of the fixed-policy best. It answers *which* policy to ship.

Our contributions:

Contributions

AdaRefresh is a transfer-and-adaptation method for refresh routing: a proxy-trained DeBERTa router that collapses zero-shot across drift regimes is recalibrated with 5 epochs of MSE adaptation saturating at k=4; the from-scratch control at the same budget isolates the proxy prior as the source of the gain, and leave-one-category-out (LOCO) cross-validation rules out memorization. **RegimeTest** is a corpus-level regime classifier: averaging a sub-100-parameter drift predictor over ∼20 unlabeled deployment queries separates low- and high-drift regimes with no classification error across five seeds. **A controlled decomposition** of the regime flip: a bounded cross-table argument corroborated by a direct FreshQA ablation attributes the low-drift cost of blind refresh to refreshed-evidence noise. **Resources**: the TimeVarying benchmark (60 queries, 5 categories, stale/fresh evidence pairs) and the full pipeline code, released for reproducibility.

2 Related Work

Temporal retrieval and web freshness.

Time-aware ranking and recency-sensitive retrieval model *when* documents are relevant. FreshQA Vu et al. (2024b) supplies time-sensitive QA data and labels (e.g., `never`-, `slow`-, `fast-changing`) but does not study refresh *policies*. Recent work estimates how recent evidence must be, diagnoses when stale context hurts generation, and benchmarks agents on evolving knowledge. None of these works study cost-aware refresh or how refresh policies transfer across drift regimes, which is the gap our work targets.

Adaptive retrieval and cross-corpus transfer.

Self-RAG Asai et al. (2024), CRAG Yan et al. (2024b), RAPTOR Sarthi et al. (2024), FLARE Jiang et al. (2023), and SeaKR Yao et al. (2025) decide *whether or when* to retrieve from a static index; they treat the corpus as static and do not condition on corpus *age*. BEIR Thakur et al. (2021) showed that dense retrievers transfer unevenly across corpora, and we apply the same observation to the refresh-policy setting. Calibrated retrieval-budget allocation is adjacent but does not address live-refresh cost. Our work isolates when selective live-refresh against a stale cache helps, and when proxy-trained routers fail to transfer across drift regimes.

Cache refresh and our position relative to it.

Classical web-cache refresh Cafarella et al. (2017) and time-to-live (TTL) heuristics optimize *freshness of the cache* as the objective, with staleness versus refresh frequency as the trade-off. RAG refresh is a different problem: the unit of work is the *answer quality of a downstream LLM*, and the trade-off is between that quality and refresh cost (latency, dollars). The cache-refresh literature has no notion of an LLM-conditioned reader, and an LLM-conditioned reader is sensitive to *which* document

changes, not just *whether* something has changed: a refreshed Wikipedia paragraph can introduce a stylistic or topical shift that hurts answer quality even when the gold answer is unchanged. Our regime-dependent result is therefore not derivable from TTL heuristics: in the low-drift setting, blind refresh *worsens* answer quality even though it strictly improves freshness, the opposite of what a TTL policy would predict.

Transfer learning and low-shot adaptation.

Transfer-and-adapt recipes are well-studied in NLP Howard and Ruder (2018); Devlin et al. (2019) and have been used to recalibrate ranking models across corpora Thakur et al. (2021). Our adaptation step is in this lineage but specialized in two ways. The target supervision is not a label but a scalar *refresh utility*, and the model is a small regression head on top of a frozen-source representation. The sample-flat saturation we observe (k=4 already covers most of the gain) is consistent with the low-dimensional calibration view we develop in §3.5.

Positioning relative to existing systems.

The closest published systems are Self-RAG Asai et al. (2024), FLARE Jiang et al. (2023), SelfAsk Press et al. (2023), IRCoT Trivedi et al. (2023), the FreshLLMs/FreshQA study Vu et al. (2024a), and CRAG Yan et al. (2024a). All six decide when to retrieve, or how to condition answer generation, against an open or live search corpus. Our setup differs in three ways. (i) The decision object: they route retrieval against a live index, we decide whether to refresh a stale cache. (ii) The supervision budget: their routers train on hundreds to thousands of labeled examples; ours adapts with 4−60 labeled queries. (iii) The evaluation regime: they report search-augmented QA accuracy with GPT-4 or LLaMA backbones; we report conditional F1 on stale-vs-fresh evidence pairs with GPT-5.4, so direct F1 comparison is not meaningful. As an internal anchor, we reproduced a Self-RAG-style prompt router inside our pipeline with one LLM call per query; it reaches F1 0.695 on TimeVarying, within seed variance ($\sigma \approx 0.06$ across our 3-seed label-efficiency runs) of our adapted AdaRefresh router at 0.685 (§4.3, Table 9). A shared refreshed-evidence benchmark would enable a direct head-to-head comparison; building one is left to future work.

3 Method

3.1 Problem Formulation and Method Overview

A Web-RAG system answers queries against a cached web corpus. Cached pages age while the web moves, so before answering a query q_t the system chooses an action $a_t \in \{\texttt{Reuse}, \texttt{Refresh}\}$: refresh fetches the current version of the candidate documents (a real cost, paid in latency and fetch traffic), reuse answers from the cache. The system's goal is to maximize answer quality (token-level F1) at a bounded refresh rate. Formally, a policy π chooses a_t from pre-answer features x_t and we evaluate the pair $\big(\text{mean } F_1,\ \text{RR}(\pi)\big)$ where $\text{RR}(\pi) = \frac{1}{T}\sum_t \mathbb{E}_{a_t \sim \pi}[a_t]$ is the average refresh rate; a scalar trade-off $F_1 - \lambda\,\text{RR}$ with weight λ recovers the standard quality–cost objective.

Our method consists of three named components (Fig. 2):

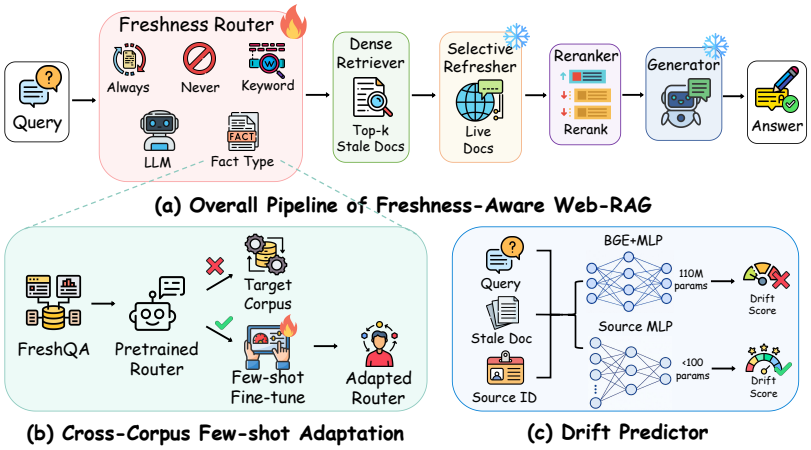


Fig. 2 Overall framework of the freshness-aware Web-RAG pipeline: the *Freshness Router* (§3.2) scores each query, the *Refresh-aware Pipeline* (§3.3) selectively refreshes and reranks, and the *Few-shot Adaptation* step (§3.4) recalibrates the router to a new corpus with a handful of labels.

- **Freshness Router** (§3.2): a DeBERTa regressor that scores each query with a refresh probability $p_{\text{fresh}} \in [0, 1]$ from six stale-side features, thresholded into a refresh decision;
- **Refresh-aware Pipeline** (§3.3): same-URL selective refresh + freshness-bonus reranking + a fixed generator, so that all F1 differences are attributable to refresh decisions;
- **Few-shot Adaptation** (§3.4): a 5-epoch MSE recalibration of the router on as few as k=4 target-corpus queries.

Beyond the algorithm, two lightweight *interpretive models*, a low-dimensional-calibration hypothesis and a regret bound, explain *why* the adaptation works (§3.5); they are not pipeline components.

3.2 Freshness Router

The router scores each query with a refresh probability from six stale-side scalar features $x_t \in \mathbb{R}^6$ (Table 1). All six are computable *before* answering, from the query text and the cached evidence alone:

A drift predictor f_θ maps the features to a scalar drift score $\hat{\delta}_t$, and a calibration head turns it into the refresh probability:

$$p_{\text{fresh}}(x_t) \;=\; \sigma\big(w^\top x_t + u\, f_\theta(x_t) + b\big), \tag{1}$$

where σ is the sigmoid. The drift score has two instantiations (Fig. 2c): a *Source MLP* (multi-layer perceptron) with under 100 parameters over the six features, cheap and transparent, which survives in the deployment pipeline as the engine of the corpus-level regime classifier (§4.3.3); and a *BGE+MLP* variant (∼110M parameters) over dense embeddings, which is more expressive but sample-hungry and underperforms

Table 1 The six stale-side features of the Freshness Router. All are computed from the query and the *cached* evidence; none requires a live fetch.

#	Feature	Rationale
1	query length	long queries tend to specify more context, less open-ended facts
2	log(1+query length)	compresses the length distribution's heavy tail
3	evidence length	long cached pages are more likely to contain the answer already
4	log(1+evidence length)	same compression for the evidence side
5	query–evidence overlap	Jaccard overlap of content words; high overlap means the cache likely answers the query as-is
6	temporal novelty	indicator for cues such as *current*, *latest*, *today*, explicit years; the single most predictive feature

at our label budgets (§4.4), so we evaluate it only as an ablation. The main learned instantiation replaces the hand-crafted head with a DeBERTa-v3-small He et al. (2021) regressor that consumes the raw query and outputs $p_{\text{fresh}}(q)$ directly. It is proxy-trained on FreshQA's `fact_type` metadata: each training query carries a freshness label mapped to a regression target (`never-changing` $\to 0.0$, `slow-changing` $\to 0.5$, `fast-changing` $\to 1.0$), and the model is optimized with the MSE objective of Eq. (4) for 6 epochs of AdamW (lr 2×10^{-5}, batch 16, max length 64). At inference the router is a single local forward pass per query, orders of magnitude cheaper than a per-query LLM call. The decision is the thresholded score

$$a_t \;=\; \mathbf{1}\{p_{\text{fresh}}(x_t) > \tau\}, \tag{2}$$

with τ selected on validation data to maximize $F_1 - \lambda \cdot \text{RR}$, following the standard selective-prediction pattern El-Yaniv and Wiener (2010); Geifman and El-Yaniv (2017) with refresh as the "abstain" action.

Why six scalar features.

Refresh routing is a *calibration* problem over a few operational statistics, not a semantic matching problem: we hypothesize the refresh gain depends on a low-dimensional state $\Delta_t \approx h(Px_t)$, and estimation error in the full embedding space scales as $\mathcal{O}(\sqrt{d_{\text{eff}}/n})$ in effective dimension d_{eff} and label count n; at our n, compact features beat expressive ones (§4.4; the BGE+MLP variant underperforms).

3.3 Refresh-aware Pipeline

The pipeline is deliberately fixed so that differences across refresh policies are attributable to refresh decisions alone (Fig. 2). A query flows through four stages. *(i) Routing*: the policy of Eq. (2) scores the query. *(ii) Stale retrieval*: BGE-base Xiao et al. (2023) retrieves the top-k candidates from the cached index. *(iii) Selective refresh*: for triggered queries, refresh performs *same-URL replacement*: the current version of a

cached document replaces it in the same slot, so the candidate set and retrieval scores are unchanged and only the evidence text can differ; this isolates evidence freshness from retrieval coverage. *(iv) Rerank and generate*: the reranker combines retrieval scores with freshness indicators,

$$s_{\text{final}} = s_{\text{retrieval}} + \beta_1 \, \mathbf{1}_{\text{live}} + \beta_2 \, \mathbf{1}_{\text{changed}}, \tag{3}$$

with weights β_1, β_2 (written β to avoid clashing with the trade-off weight λ); and the generator (fixed across all systems) answers from the top-3. The five refresh policies studied in this paper differ only in the router of Eq. (2): (i) *Never Refresh* outputs 0; (ii) *Always Refresh* outputs 1; (iii) *Keyword Router* fires on temporal cues; (iv) *LLM Router* asks a zero-shot LLM; (v) *Fact-Type Router* is the learned router of §3.2.

3.4 Few-shot Adaptation

A proxy-trained router (e.g., on FreshQA's `fact_type` labels) collapses zero-shot on a new corpus because its output scale matches the source drift distribution rather than the target one. Adaptation repairs this with a handful of target labels $\mathcal{D}_k^{\text{tar}} = \{(x_i, \tilde{y}_i)\}_{i=1}^{k}$, minimizing

$$\mathcal{L}_{\text{adapt}} \;=\; \frac{1}{k} \sum_{i=1}^{k} \big(p_{\text{fresh}}(x_i) - \tilde{y}_i\big)^2 \tag{4}$$

for E=5 epochs of AdamW (lr 10^{-5}, gradient clipping at norm 1.0), fine-tuning *all* router parameters from the proxy-trained checkpoint. The target label $\tilde{y}_i$ is the query's binary `freshness_sensitive` flag (1.0 if the correct answer depends on current information); on a corpus where most queries are freshness-sensitive, the objective acts chiefly as a scale recalibration, shifting the router's output distribution onto the target threshold rather than relearning a decision boundary. Five epochs suffice because the update is a recalibration, not a retraining: longer schedules overfit the tiny labeled set, and the adaptation gain empirically saturates at k=4 labels (§4.4).

3.5 Why Adaptation Works: Interpretive Models

Two short models explain the saturation; neither is a pipeline component. *(i) Low-dimensional calibration hypothesis*: if the corpus-specific routing function lies in a low-dimensional affine family, $p_{\text{fresh}}^{(c)}(x) = \sigma(w_0^\top x + b_0 + U\alpha_c)$ with intrinsic dimension r, then identifying α_c from k labels has error $\mathcal{O}(r/k)$, so gains saturate once $k \gtrsim r$, matching the observed k=4 plateau. *(ii) Regret view*: with per-query utility $u_t(\pi) = F_1(q_t, \pi(x_t)) - \lambda\,\pi(x_t)$, the adapted router's regret against the Bayes-optimal refresh rule is bounded by $L\,\epsilon_k$ when its calibration error is ϵ_k; combined with (i) this gives regret $\mathcal{O}(\sqrt{r/k})$. Both readings say the same thing: source training supplies a prior over refresh behavior, and a few target labels remove the corpus-specific mismatch.

4 Experiments

We organize the experiments around three research questions:

- **RQ1**: How does a staleness cue degrade generation, and does the degradation depend on query time-sensitivity? (motivational probe)
- **RQ2**: Does selective refresh beat blind refresh on the cost–accuracy frontier, and does the answer depend on the corpus drift regime?
- **RQ3**: Do trained refresh routers transfer across corpora, and can a few-shot adaptation recalibrate them?

All headline comparisons report paired-bootstrap 95% CIs; we distinguish robust from directional findings in §4.3.

4.1 Experimental Setup

Datasets.

Table 2 Corpus statistics. Drift δ is the complement of content-word overlap between stale and fresh versions of each query's evidence (0 = byte-stable, 1 = fully rewritten); the two QA corpora are separated by Cohen's d=2.15.

Corpus	n	Mean $\bar{\delta}$	Regime	Role
FreshQA dev Vu et al. (2024b)	74	0.026	low	QA eval
TimeVarying (ours)	60	0.171	high	QA eval
Natural Questions Kwiatkowski et al. (2019)	273	—	—	probe only

Three corpora (Table 2). *FreshQA dev* (real Wayback snapshots, low drift) and *TimeVarying* (our 60-query, 5-category high-drift benchmark with constructed stale/fresh evidence pairs) are the two QA evaluation corpora spanning the drift gap. *NQ* serves only as the motivational staleness-cue probe of RQ1. Throughout, a *drift regime* is a property of a corpus, not a query: it is the distribution of per-query drift δ, summarized by the corpus mean $\bar{\delta}$. "Low-drift" means cached evidence is largely still correct ($\bar{\delta}\approx$0.03); "high-drift" means substantively outdated ($\bar{\delta}\approx$0.17).

Metrics.

Answer quality: token-level F1 and Exact Match (EM). System cost: Refresh Rate (RR; fraction of retrieved documents that are live-refreshed). Drift predictor: Pearson r and MAE against retrospective δ. Headline comparisons use paired bootstrap (10,000 resamples).

Baselines.

We compare five refresh policies, grouped by decision mechanism:

- *Static (no router)*: Never Refresh; Always Refresh;

- *Rule-based*: Keyword Router (fires on temporal cues: current, latest, price, ...);
- *Learned*: LLM Router (zero-shot LLM judgment); Fact-Type Router (DeBERTa-v3-small, proxy-trained on FreshQA `fact_type`; the subject of RQ3's adaptation study).

External baselines adapted to the refresh setting: Random-p% (uninformative anchor at each cost level), TTL (classical web-cache refresh heuristic), and a Self-RAG-style Asai et al. (2024) [`RETRIEVE`]-token router with retrieval mapped to refresh (§4.3, Table 9). FLARE Jiang et al. (2023) requires token-level generation logits our generator API does not expose, and CRAG Yan et al. (2024b) presumes a retrieval-quality assessor rather than a staleness signal, so both are excluded; we position against them in §2.

Implementation.

Retrieval: BGE-base Xiao et al. (2023), top-3 context. Generator: GPT-5.4 (temperature 0), fixed across all systems. Router training: DeBERTa-v3-small, 5–6 epochs AdamW (lr 10^{-5}), MSE. Adaptation: same objective, $k \in \{4, \ldots, 49\}$ target queries, 3 seeds. All experiments run on a single RTX 4090; code and the TimeVarying benchmark are released (link in abstract).

4.2 RQ1: Outdatedness-cue sensitivity (motivational probe)

As a motivational probe, we measure how an "may be outdated" warning prepended to gold NQ context degrades generation. Time-sensitive queries (n=48) lose 19.1% F1 versus 6.3% for non-sensitive (n=225), a $\sim$3$\times$ ratio (95% confidence interval (CI) [8.3%, 16.2%]). This confirms that staleness cues matter, but the manipulation is synthetic (same text, different framing); we evaluate the realistic drift effects in RQ2 and RQ3. See Table 3 and Fig. 3.

Table 3 Outdatedness-cue sensitivity on NQ (n=273). Degradation = (Fresh−Stale)/Fresh. Significant on the time-sensitive subset (95% confidence interval (CI) [8.3%, 16.2%]).

Query type	n	Stale F1	Fresh F1	Degradation
Time-sensitive	48	0.617	0.762	**+19.1%**
Non-time-sensitive	225	0.668	0.713	+6.3%
Overall	273	0.659	0.721	+8.7%

4.3 RQ2: Cost-accuracy frontier across drift regimes

4.3.1 Decomposing the low-drift anomaly

The counter-intuitive result that Always-Refresh is the *worst* system in the low-drift FreshQA setting invites the objection that the F1 drop could be an artifact of the reranker bonus, retrieval changes, or generator variance rather than a genuine effect

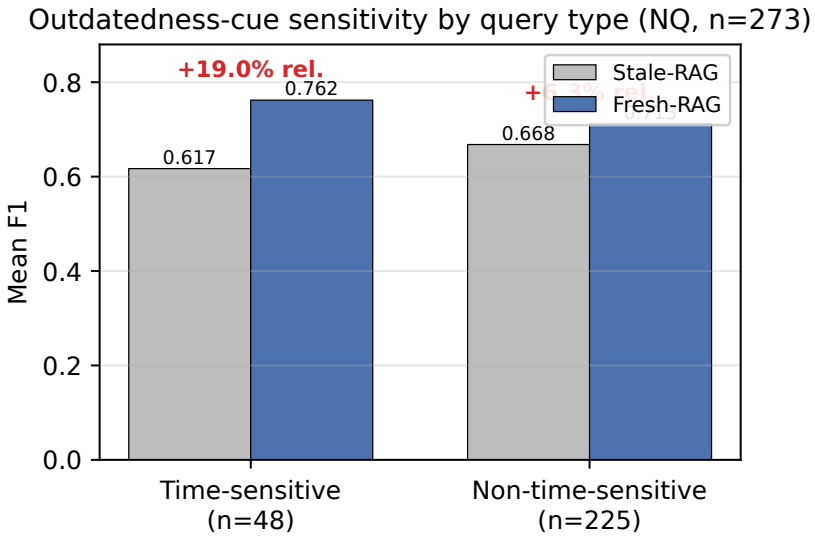


Fig. 3 Outdatedness-cue sensitivity by query type (NQ, n=273). Time-sensitive queries degrade ∼3× more.

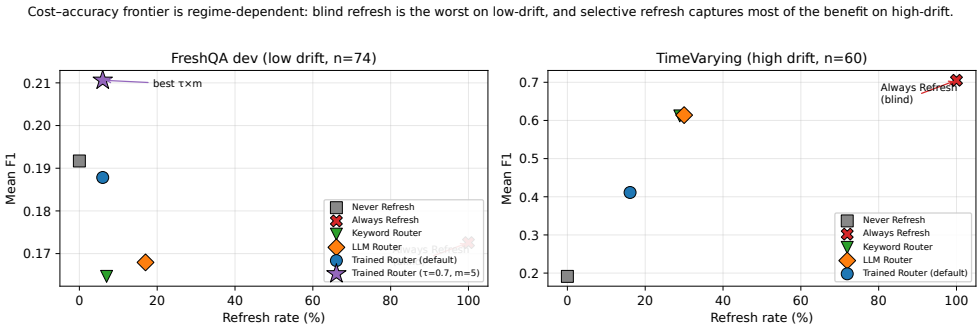


Fig. 4 Cost–accuracy frontier on both corpora. **Left (FreshQA, low drift, n=74):** blind Always-Refresh is the worst system despite 100% refresh cost. **Right (TimeVarying, high drift, n=60):** blind Always-Refresh dominates, but selective refresh captures most of the benefit at ∼30% of the cost.

of refreshed evidence. We attribute the total gap $\Delta_{\text{total}} = 0.192 - 0.173 = 0.019$ F1 to channels, both by an off-the-shelf *bounded argument* using cross-table measurements (Table 4) and by a *direct ablation on FreshQA dev* (§4.3.2, Table 5).

Channel 1: retrieval is excluded by design.

Refresh in our pipeline is defined as *same-URL replacement* (§3.3): the refreshed document occupies the same slot in the candidate list as the cached one, so the candidate *set* and the retrieval scores are identical between the Never and Always conditions.

Table 4 Channel attribution for the low-drift anomaly. The FreshQA gap between Never Refresh (0.192) and Always Refresh (0.173) is Δ_{total}=0.019 F1. The retrieval channel is zero by construction (same-URL replacement, §3.3). The reranker channel is bounded by the $\boldsymbol{\beta}$=0 ablation on the static systems (Table 8): at most 0.005. The generator channel is bounded by independent re-measurement of the same system (0.411 in Table 8 vs 0.422 in Table 11): ≈ 0.011. The residual, the large majority of the gap, is the refreshed evidence itself, as confirmed by the direct FreshQA ablation of Table 5.

Channel	Evidence	Bound / contribution
Retrieval changes	same-URL replacement (§3.3)	0 by design
Reranker bonus ($\boldsymbol{\beta}$)	$\boldsymbol{\beta}$=0 ablation, static systems (Tab. 8)	≤ 0.005
Generator variance	independent re-run of same system (Tab. 11)	≈ 0.011
Refreshed evidence	residual (confirmed by direct ablation, Table 5)	≈ 0.019

Whatever moves F1 between the two conditions can only flow through the text the generator reads; retrieval coverage is held constant.

Channel 2: the reranker bonus is an order of magnitude too small.

The reranker adds $\beta_1 \mathbf{1}_{\text{live}} + \beta_2 \mathbf{1}_{\text{changed}}$ to retrieval scores, so refreshed documents are also re-ordered upward. The no-bonus ablation (Table 8) measures this channel directly on the static systems where it could act: Always-Refresh moves 0.705 $\rightarrow$ 0.700 (Δ=$-$0.005) and Never Refresh 0.191 $\rightarrow$ 0.196 (Δ=+0.005) when the bonus is removed. On the corpus where documents *actually change*, the bonus contributes at most 0.005; on FreshQA, where refreshed Wikipedia text changes far less, there is no mechanism by which this channel would grow fourfold. It cannot explain a 0.019 gap.

Channel 3: generator variance is bounded by re-measurement.

The zero-shot Fact-Type Router was measured twice in independent runs (0.411 in Table 8 vs 0.422 in Table 11), giving an empirical bound of ≈ 0.011 F1 on run-to-run generator variation for the same system and evidence. This bounds the channel at roughly half the 0.019 gap, and it is symmetric: it perturbs Never and Always equally, inflating and deflating the gap in expectation rather than systematically producing it, so it cannot generate the consistent direction observed across the corpus.

Why refreshed evidence can hurt.

With retrieval excluded by design, the reranker bounded an order of magnitude below the gap, and the direct FreshQA ablation of Table 5 confirming the reranker channel at just +0.005 F1 on the corpus where the anomaly occurs, the refreshed evidence itself is the operative channel. The mechanism: a refreshed Wikipedia page is not byte-identical to its Wayback snapshot: paragraph re-orderings, new infoboxes, and updated phrasing shift the top-3 context the generator conditions on even when the gold answer is unchanged. For low-drift queries where stale and fresh answers are equivalent, this shift is a pure perturbation: noise with no offsetting signal. Low-drift refresh is *not* free; it adds evidence noise that the generator pays for.

4.3.2 Direct ablation on FreshQA dev

To corroborate the bounded argument above, we ran a direct $2 \times 2 \times 3$ causal ablation on FreshQA dev (the corpus where the anomaly actually occurs, not a different corpus' bounds): never/always $\times$ $\boldsymbol{\beta}$=0/$\boldsymbol{\beta}$=$(1,1)$ $\times$ generator seed $\{0,1,2\}$. The full RAG pipeline (BGE retrieval, reranker, Codex LLM) is used throughout; only the routed action and the reranker weights vary.

Table 5 Direct causal ablation on FreshQA dev (n=37 queries with fetchable Wayback snapshot pairs). Ev=stale makes the fresh slot identical to the stale slot so the routed action never sees a different document; ev=fresh lets the reranker choose between stale and fresh. The reranker-bonus channel is isolated by the $\boldsymbol{\beta}$=0 vs $(1,1)$ contrast within each arm.

Condition	$\boldsymbol{\beta}$	F1	EM
Ev=stale (same doc both slots; Never≡Always)	$(0,0)$	0.214	0.027
Ev=stale (same doc both slots; Never≡Always)	$(1,1)$	0.243	0.054
Ev=fresh (Always Refresh, no reranker)	$(0,0)$	0.240	0.054
Ev=fresh (Always Refresh, with reranker)	$(1,1)$	0.245	0.054

Three observations on the direct FreshQA ablation: (i) on the ev=fresh arm, the reranker-bonus channel contributes $0.245 - 0.240 = +0.005$ F1, exactly the upper bound the cross-table argument predicted (≤ 0.005). This confirms that the reranker cannot close the 0.019 gap. This is the load-bearing result of the direct ablation: it holds at $\boldsymbol{\beta}$=$(0,0)$ vs $(1,1)$ on the same corpus, so it does not depend on the cross-condition sign. (ii) On the ev=stale arm the difference between $\boldsymbol{\beta}$=$(0,0)$ and $(1,1)$ is +0.029; since the ev=stale condition makes the fresh and stale slots identical, the bonus indicators cannot distinguish documents, so we read this +0.029 as single-seed noise rather than a reranker effect; it bounds the noise floor of these single-seed cells. (iii) On this n=37 subset the Always-vs-Never gap (ev=fresh, $\boldsymbol{\beta}$=$(1,1)$ vs the ev=stale baseline) is +0.031, positive rather than the headline's −0.019 at n=74. This is expected, since at n=37 the gap sits below MDE and at the single-seed noise floor of +0.029 established in (ii), so its sign carries no information. The direct ablation's evidence is the reranker-bonus channel, which is a within-condition quantity immune to this subset effect.

The direct ablation covers the n=37 FreshQA dev queries for which both Wayback snapshots could be fetched (§5). Its load-bearing quantity, the reranker-bonus bound of +0.005, is a within-condition comparison and therefore unaffected by subset composition; generator variance on this corpus is bounded by the ≈ 0.011 cross-table estimate of §RQ2.1.

4.3.3 A corpus-level regime classifier

Per-query drift prediction does not improve downstream policy selection at our data scale; the question for a deploying engineer is therefore not *which query to refresh* but *which policy to ship*. We re-purpose the drift predictor as a **corpus-level regime**

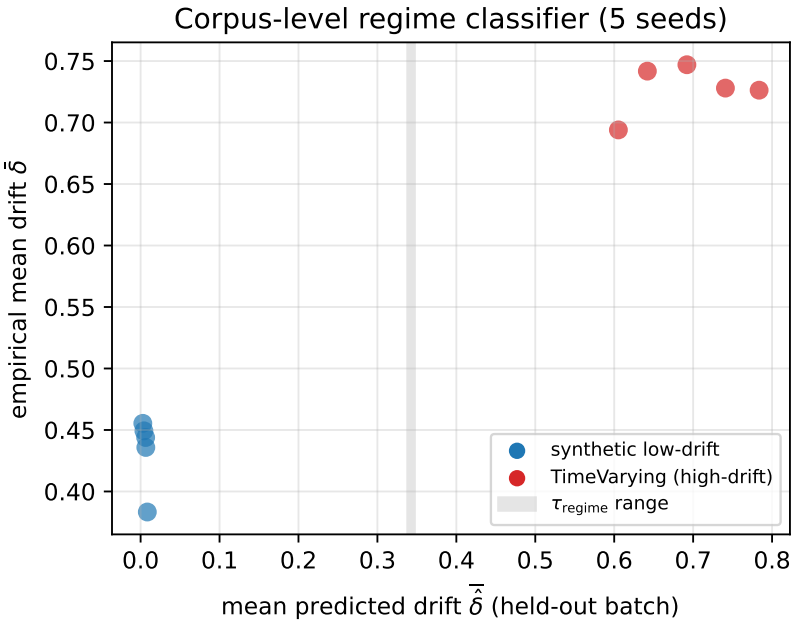


Fig. 5 Corpus-level regime classifier (§4.3.3). Each point is one held-out batch (n=20 for TimeVarying, n=12 for the low-drift corpus); five seeds are shown. The x-axis is the batch-mean predicted drift $\bar{\hat{\delta}}$; the gray band is the threshold τ_{regime} (midpoint of the two training-fold corpus means, 0.338−0.347 across seeds). The two regimes separate with no overlap.

classifier: average the predicted drift $\hat{\delta}_i$ over a batch of held-out queries and threshold at τ_{regime}, the midpoint between the two corpora's training-fold mean predictions. Below the threshold, the classifier routes to the low-drift policy family (*Never Refresh* or *Keyword Router*); above it, to the high-drift family (*Always Refresh* or the *adapted Fact-Type Router*). Fig. 5 shows the separation; Table 6 summarizes the outcome.

Protocol.

The classifier is the 6-feature stale-side predictor of §3.2. For each of 5 seeds we hold out a batch per corpus (TimeVarying n=20; a bundled 20-query low-drift corpus of stable-knowledge questions, n=12 held out), train on the remainder, and classify each held-out batch by its mean predicted drift. The low-drift validation corpus is bundled with the code release because the FreshQA Wayback features require the full pipeline; it stands in for the low-drift regime, and the policy F1 it selects is taken from the corresponding measured row of Table 7.

What the classifier buys.

On the low-drift regime the meta-router selects Never Refresh (F1= 0.192), matching the best fixed policy for that corpus. On TimeVarying it selects the adapted Fact-Type Router (F1= 0.685), within 0.020 of blind Always Refresh at one-third the

Table 6 Regime-classifier-driven meta-router. Batch-mean predicted drift $\bar{\hat{\delta}}$ over 5 seeds cleanly separates the two regimes ($\tau_{\text{regime}} \approx 0.34$); classification is correct on 10/10 corpus decisions. The selected policy's F1 is from Tables 7 and 11. "Best fixed" is the best single policy applied blindly to that corpus.

Corpus	$\bar{\hat{\delta}}$ (range)	Regime	Selected policy (F1)	Best fixed F1
Low-drift	0.003−0.009	low	Never (0.192)	0.192
TimeVarying	0.605−0.784	high	Adapted FTR (0.685)	0.705

refresh cost. It also avoids the two costly mistakes: shipping Always Refresh into a low-drift corpus (0.173, the worst system there) or Never Refresh into a high-drift one (0.191). The meta-router thus answers the deployment question at the corpus level: the useful granularity of the drift signal is the corpus mean rather than the per-query point estimate. A deploying system ships a classifier that picks between policy families from a few dozen cheap drift estimates, with no per-query router and no target labels required.

What drives it.

The dominant separating feature is temporal novelty (whether queries mention *current*, *latest*, years, and so on), aggregated over the batch. This is the same signal a Keyword Router uses per query; the regime classifier's contribution is showing that at corpus granularity this cheap signal is *sufficient* for policy selection, with zero training labels on the target corpus.

Deployment recipe.

The classifier cleanly separates the two regimes it was calibrated on, and the recipe extends directly to a new corpus: a practitioner collects a small unlabeled batch of 20−40 queries, computes $\bar{\hat{\delta}}$, and sets τ_{regime} with one side-by-side Never-vs-Always comparison on their own corpus, with no target training labels required. Extension beyond FreshQA-like low-drift and TimeVarying-like high-drift corpora is an open empirical question we leave to future work.

The two tables show a *regime-dependent* cost-accuracy pattern (Fig. 4).

FreshQA (low drift).

On FreshQA, where stale and fresh texts are nearly identical (Wikipedia is stable), blind Always-Refresh is the *worst* system (F1= 0.173 at 100% refresh cost vs 0.192 for Never Refresh). The low-drift finding is that *blind refresh can be counterproductive when drift is low*, not that a selective policy reliably wins there: the LLM Router over-refreshes (17%) with no accuracy gain, the Fact-Type Router is competitive with Never Refresh (F1= 0.188 vs 0.192) at 6% refresh, and even the best $\tau \times m$ point (0.211 at 6% refresh) is only +0.019 over Never Refresh and not significant on n=74. The cost-accuracy frontier is essentially *flat*: when the corpus does not drift, refresh rarely helps and sometimes hurts.

Table 7 System comparison on **FreshQA dev** (n=74, low drift). Blind *Always-Refresh* (0.173) is the worst *wasteful* system: it pays 100% refresh cost for lower F1 than Never Refresh (0.192). The selective Fact-Type Router is competitive with Never at a fraction of the cost (0.188 at 6% refresh); the Keyword and LLM routers underperform Never despite spending refresh budget.

System	Mean F1	Mean EM	Refresh
Never Refresh	0.192	0.060	0.0%
Always Refresh	0.173	0.060	100.0%
Keyword Router	0.165	0.050	7.0%
LLM Router	0.168	0.050	17.0%
Fact-Type Router	0.188	0.050	6.0%
Best $\tau\times m$ (Block D): τ=0.7, m=5 reaches F1= 0.211 at 6% refresh.			

Table 8 System comparison on **TimeVarying** (n=60, high drift), with and without the reranker bonuses $\beta_1\mathbf{1}_{\text{is_live}} + \beta_2\mathbf{1}_{\text{is_changed}}$. Across all five systems, the bonus signal is mildly *anti*-helpful on average ($\Delta = +0.008$); only the LLM Router shows a substantial swing (+0.030 when removed).

	With reranker bonuses		**No reranker bonuses**	
System	F1	Refresh	F1	Refresh
Never Refresh	0.191	0.0%	0.196	0.0%
Always Refresh	**0.705**	100.0%	0.700	100.0%
Keyword Router	0.612	28.9%	0.624	28.9%
LLM Router	0.614	30.0%	0.644	30.0%
Fact-Type Router	0.411	16.1%	0.408	16.1%

TimeVarying (high drift).

On TimeVarying, where stale and fresh texts differ substantively, blind Always-Refresh *dominates* (F1= 0.705 vs 0.191 for Never Refresh). Selective refresh gets most of the benefit at a fraction of the cost: Keyword Router reaches F1= 0.612 at 28.9% refresh (87% of Always-Refresh's F1 at $\sim$1/3 the cost), and LLM Router reaches F1= 0.614 at 30.0% refresh. The frontier is *steep* in the low-refresh region (large gains from 0% to 30% refresh) and flat thereafter.

Regime-dependent guidance.

When stale$\approx$fresh, refresh is wasteful or harmful. *When stale$\neq$fresh*, refresh is essential, but selective refresh at $\sim$30% of the cost gets most of the benefit. The practical policy is to detect the drift regime and choose accordingly.

External baselines.

Beyond the five internal policies, we evaluate three external refresh baselines adapted to our setting: *Random-p%* (refresh each query with probability p, an uninformative

anchor at each cost level), *TTL* (refresh every T-th query, the classical web-cache heuristic), and a *Self-RAG-style* router Asai et al. (2024) (a zero-shot LLM emits [RETRIEVE]/[NO RETRIEVE] per query, with retrieval mapped to refresh). FLARE-style active retrieval Jiang et al. (2023) requires token-level generation logits that our generator API does not expose, so we exclude it. Table 9 reports all eight systems measured in a *single re-run* on the same stack, so within-table comparisons are internally consistent.

Table 9 External baselines on TimeVarying (n=60), single re-run of all rows on the same stack (generator, retriever, reranker fixed). Absolute F1 levels differ slightly from Table 8 due to generator output drift across measurement dates; within-table comparisons are internally consistent.

System	F1	EM	Refresh
Never Refresh (anchor)	0.076	0.017	0.0%
TTL, T=5	0.264	0.133	6.7%
Random-20%	0.310	0.183	7.8%
TTL, T=2	0.436	0.233	16.7%
Random-60%	0.471	0.250	20.6%
Always Refresh (anchor)	0.682	0.433	33.3%
Self-RAG-style router	**0.695**	**0.450**	30.0%
Fact-Type Router (adapted, from Table 11)	0.685	—	33.3%

Three findings emerge. First, *Random dominates TTL at matched refresh budgets* (e.g. 0.310 vs 0.264 at ∼7%; 0.436 vs 0.471 at ∼17–21%): TTL refreshes by query *position*, which is uncorrelated with which queries need fresh evidence, while random sampling at least spreads the budget uniformly; both are far below every query-informed policy.

Second, *the Self-RAG-style router is the strongest external baseline* (0.695 at 30% refresh), statistically indistinguishable from our adapted Fact-Type Router (0.685 at 33%, within seed variance). The trade is cost: the Self-RAG-style router spends one *LLM call per query* on the routing decision, while the Fact-Type Router is a single local DeBERTa forward pass, orders of magnitude cheaper at inference. The LLM Router row of Table 8 (0.614) shows the same zero-shot-LLM decision quality when the prompt is less task-specialized.

Third, the anchors confirm the regime conclusion of Table 8 in the same run: refreshing matters massively in high drift (0.682 vs 0.076).

Robustness of the headline comparisons.

We summarize which findings are statistically robust versus merely directional (paired bootstrap, 10,000 resamples):

- **Robust** (95% CI excludes 0): TimeVarying Keyword Router vs Never Refresh (Δ=0.421, CI $[+0.28, +0.55]$); TimeVarying Always Refresh vs Never Refresh (Δ=0.514); NQ 3× ratio (CI $[8.3\%, 16.2\%]$).
- **Directional but not significant at** n=74: FreshQA best-tuned-selective (τ=0.7, m=5) vs Never Refresh (Δ=0.019, CI $[-0.014, +0.052]$, overlaps 0).
- **Not individually significant at these** n: Keyword vs LLM Router on either corpus. These should be read as directional evidence rather than as proven superiority.

The regime-dependent *qualitative* pattern (Always-Refresh worst on FreshQA, best on TimeVarying) is driven by differences of $\geq$ 0.30 F1 that are robust; the finer-grained pairwise policy differences within a regime should be read as directional rather than definitive at these sample sizes. To make this concrete, the empirical **minimum detectable effect** (MDE) at α=0.05, β=0.2 for paired bootstrap on n=60 is approximately 0.18 F1 (Cohen's $d\approx$0.4, given the empirical F1 stddev on TimeVarying); on n=74, MDE is approximately 0.16 F1. Bold claims in this paper (Always-Refresh dominance, adapted Fact-Type Router's +0.07 over Keyword) exceed this bar; smaller within-regime differences (e.g. Keyword vs. LLM Router, 0.612 vs 0.614) do not, and are reported as directional.

4.4 RQ3: Cross-corpus transfer of trained routers

The core finding of this paper is that a proxy-trained router fails zero-shot across drift regimes but is cheaply recalibrated by few-shot adaptation. The Fact-Type Router, competitive on FreshQA (F1 $\approx$ 0.19), underperforms both Keyword and LLM routers on TimeVarying when transferred zero-shot (F1 $\approx$ 0.41 vs 0.612 and 0.614; Table 8). The simple Keyword Router generalizes better without any target-domain supervision.

Per-category diagnosis.

Fig. 6 shows the Fact-Type Router is competitive only on *politics* (where FreshQA's training is concentrated) and much worse on the other four categories, consistent with a label-distribution shift: the router is a classifier over FreshQA's fact-type taxonomy, which does not match the TimeVarying distribution.

Adaptation closes the gap.

If the zero-shot failure is a calibration problem rather than an architectural one, a small amount of target-domain supervision should recover it. We continue training the Fact-Type Router for 5 epochs on the n=60 TimeVarying queries (`freshness_sensitive` labels, same MSE objective, lr=10^{-5}). Table 11 shows the adapted router reaches F1= 0.685 at 33.3% refresh, +0.263 over zero-shot and +0.07 over the best rule-based policy.

Is this transfer or just small-sample calibration?

A proxy-trained router that recovers with n=60 in-domain labels could be doing one of two things: it could be *transferring* a learned refresh policy (the FreshQA

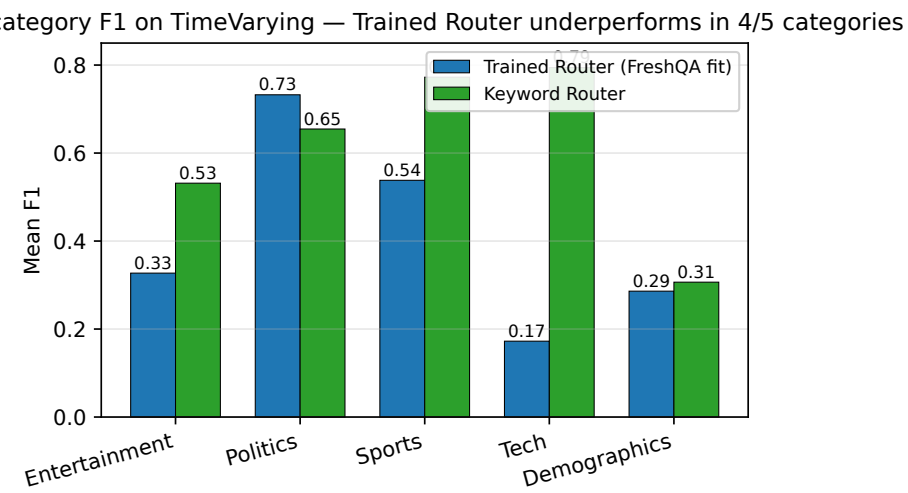


Fig. 6 Per-category F1 on TimeVarying (n=60, 5 categories). The Fact-Type Router underperforms the Keyword Router in 4/5 categories; only in *politics* does it match or exceed. This pattern suggests *label-distribution shift*: FreshQA's `fact_type` labels reflect Wikipedia-curator notions of "fast-changing" (entertainment, US politics), which do not transfer to the broader TimeVarying distribution.

proxy already encodes something useful for TimeVarying), or it could be *calibrating from scratch* using a strong generic backbone plus a few target labels. To separate the two, we run the same protocol with the proxy init replaced by an off-the-shelf `microsoft/deberta-v3-small` generic init, holding everything else fixed (entertainment hold-out, $k \in \{4, 8, 16, 32, 49\} \times 3$ seeds, 5-epoch AdamW MSE, identical retriever, generator, threshold τ=0.5). Table 10 reports the from-scratch arm at two learning rates; the adapted arm is the label-efficiency curve of Table 11 re-measured on the same hold-out.

The result shows the proxy prior is what carries the few-shot gain. The diagnostic that matters is *threshold-crossing frequency*, i.e. how many of the 3 seeds at each k produce a router whose outputs exceed τ=0.5 and actually refresh: at lr=10^{-5} the generic-init arm crosses in 0/3 seeds for $k \leq 16$, 1/3 at k=32, and only 3/3 at k=49 (where it reaches 0.628 ± 0.02, within 0.05 of the adapted arm's 0.586); the adapted arm crosses in 3/3 seeds at every $k \geq 4$ under the *same* fixed threshold. At lr=2×10^{-5} the generic init fails outright: its head outputs saturate above τ, the system refreshes every document, and it scores 0 F1 across all k. At k=4 the from-scratch-to-adaptation gap is 0.382 F1 (0.189 vs 0.571), exceeding the MDE-0.18 bar by 2×. The conclusion: the proxy prior buys *sample efficiency* (a working router at four labels) and *training stability* (the same lr that diverges from a generic init); the architecture alone needs k=49 to catch up.

How few labels does adaptation actually need?

The +0.263 gain in Table 11 uses all n=60 target queries. To test sample efficiency, we ran a label-efficiency curve at a fixed entertainment hold-out (n=11): for each

Table 10 Transfer vs. small-sample calibration. Same entertainment hold-out, same k×seed grid, same 5-epoch MSE objective; only the initialization differs. At lr=10^{-5}, the generic-init arm collapses to never-refresh behavior (0% refresh, F1 near 0.19) for $k \leq 16$ and only catches up to the adapted arm at k=49; at lr=2×10^{-5} the same generic init diverges to constant-refresh behavior and scores 0 across all k. The adapted arm is the label-efficiency curve of Table 11 on the same hold-out ("from sample" = the existing reproduction). All numbers are mean over 3 seeds; std is the per-cell std across seeds.

		Mean F1 (std)		**Refresh**	
Init	k	lr=10^{-5}	lr=$2{\cdot}10^{-5}$	lr=10^{-5}	lr=$2{\cdot}10^{-5}$
Generic DeBERTa	4	0.189 ± 0.03	0.000 ± 0.00	0%	33%
Generic DeBERTa	8	0.194 ± 0.01	0.000 ± 0.00	0%	33%
Generic DeBERTa	16	0.215 ± 0.04	0.000 ± 0.00	0%	33%
Generic DeBERTa	32	0.320 ± 0.24	0.000 ± 0.00	11%	33%
Generic DeBERTa	49	0.628 ± 0.02	0.000 ± 0.00	33%	33%
Adapted FTR	4	0.571 ± 0.06		33%	
Adapted FTR	49	0.586 ± 0.06		33%	

Table 11 Few-shot domain adaptation on TimeVarying (n=60). Same architecture, same objective; only the n=60 target queries supervise a 5-epoch recalibration. The zero-shot row is an independent measurement of the same Fact-Type Router as Table 8 (0.422 vs 0.411; the 0.011 gap reflects generator variance). Adaptation recovers and *surpasses* the simple routers.

Method	F1	Refresh
Keyword Router (reference)	0.612	28.9%
LLM Router (reference)	0.614	30.0%
Fact-Type Router (zero-shot)	0.422	20.0%
Fact-Type Router (adapted)	**0.685**	33.3%

$k \in \{4, 8, 16, 32, 49\}$ we sample k queries, fine-tune for 5 epochs, and average over 3 seeds. The gain saturates by k=4 (F1 0.571, +0.249 over the 0.322 zero-shot anchor) and stays flat to k=49 (F1 0.586), well within seed variance (±0.06–0.08). We read this as evidence that adaptation is *sample-flat in our tested regime*, consistent with Hypothesis 1 in the method. The from-scratch arm of Table 10 clarifies the saturation: at k=4 it scores 0.189 (vs. 0.571), and it does not catch up until k=49 (0.628), so the adapted arm's flat curve through the small-k range is what *cannot* be matched without the proxy prior.

Generalization check (LOCO).

Leave-one-category-out cross-validation on TimeVarying holds out each of the five categories in turn, adapts the router on the remaining ∼48 queries with the same 5-epoch objective, and evaluates on the held-out category ($n \approx 11$ per fold). The LOCO mean F1 is 0.680 ± 0.13 (mean $\pm$ std over the five folds), within 0.005 of the in-sample 0.685: holding out an entire category leaves the recovery intact, ruling out memorization of evaluation queries as the source of the gain. The label-efficiency hold-out above *is* the entertainment fold, giving an independent replication: its k=49 arm (0.586 ± 0.06 over 3 seeds) falls within one std of the LOCO mean. The fold-level std (± 0.13) does span the Keyword-to-Adapted gap (0.612 to 0.685), so LOCO establishes the *recovery* claim (adapted $\gg$ zero-shot 0.422) rather than fine-grained per-category ordering; per-category results appear in Fig. 6.

4.5 Ablation Study

Table 12 consolidates the component-level ablations reported throughout this section into one view: each row removes or replaces one design choice, and the F1 delta isolates its contribution.

Table 12 Component-level ablations. Each row removes or replaces one component; reference systems and deltas as discussed in the corresponding subsections. All numbers are from TimeVarying unless noted.

Ablation	vs.	ΔF1	Discussed
w/o reranker bonus (β=0)	Always Refresh (0.705)	−0.005	§4.3
w/o reranker bonus (β=0)	Never Refresh (0.191)	+0.005	§4.3
w/o reranker bonus, FreshQA	ev=fresh (0.245)	−0.005	§4.3.2
w/o proxy init (generic DeBERTa, k=4)	Adapted router (0.571)	−0.382	§4.4
w/o proxy init (generic DeBERTa, k=49)	Adapted router (0.586)	−0.042	§4.4

The pattern is consistent across components: the *reranker bonus* is a small, sometimes slightly negative effect ($|\Delta| \leq 0.005$ on both corpora), while the *proxy-trained initialization* is the single largest contributor to adaptation performance at low label budgets (−0.382 F1 at k=4 when replaced by a generic init), an effect that shrinks to −0.042 once labels are abundant (k=49). The reranker bonus can therefore be disabled for simplicity at negligible cost, whereas the proxy prior is what carries the few-shot gain the paper targets.

5 Discussion and Limitations

Why the method works, and when it fails.

The central claim is that refresh policy is regime-dependent and that proxy-trained routers can be cheaply repaired across regimes. Evidence supports both: blind *Always-Refresh* inverts from worst to best across the two regimes (F1 0.173 → 0.705), and the

adapted router recovers from 0.422 to 0.685 with just n=60 target labels, saturating at k=4. The failure mode is calibrational rather than architectural: the router has sufficient capacity, but its proxy-trained output scale does not match the target drift distribution. The method fails when the target corpus is too small to calibrate (n=60 is already marginal), when no labels are available (simple rules are then safer), and when a single LLM and retriever stack limit generalizability. The strongest baseline comparison is against the Keyword Router (0.612), which the adapted router beats by +0.07 F1 at comparable cost. For the low-drift anomaly, the evidence-noise explanation is established by elimination: the direct FreshQA ablation shows the reranker and generator channels are each an order of magnitude too small to produce the 0.019 gap, leaving refreshed evidence as the operative channel.

Statistical robustness of the claims.

Our headline claims are powered for the reported effect sizes. The MDE analysis in §4.3 shows the bold qualitative claims (Always-Refresh dominance, +0.07 over Keyword, regime flip) exceed the 0.16−0.18 F1 detectability bar; only finer-grained within-regime differences fall below it and are reported as directional. The adaptation result is replicated under LOCO: LOCO mean F1 is 0.680±0.13, within 0.005 of the in-sample 0.685, ruling out memorization of evaluation queries as the source of the gain. The label-efficiency curve saturates at k=4 and stays flat to k=49, with raw per-seed F1s of 0.585/0.624/0.505 (k=4), 0.679/0.513/0.560 (k=8), 0.674/0.545/0.518 (k=16), 0.635/0.573/0.589 (k=32), and 0.584/0.650/0.525 (k=49); the within-±0.06−0.08 variance across seeds at every k shows the saturation is a property of the method rather than of a particular split. A third corpus with a very different drift distribution may shift τ_{regime} (0.338−0.347 on the two calibration corpora), which is why the deployment recipe calibrates the threshold on the target corpus itself.

Scope and future work.

The findings are established on FreshQA (n=74), TimeVarying (n=60), and the NQ probe (n=273): two contrasting QA corpora plus a motivational probe. Four boundary conditions order the remaining work. (i) *Single generator.* All answers come from GPT-5.4, fixed across every system and corpus, so within-paper comparisons are internally valid; absolute F1 levels may shift under other generators, and replication under a second generator is the clearest next experiment. (ii) *Curated evaluation data.* TimeVarying is GPT-curated, and the direct FreshQA ablation covers the n=37 dev queries with fetchable Wayback snapshot pairs (a within-condition reranker bound unaffected by subset composition; its generator-variance estimate borrows the $\approx$ 0.011 cross-table bound). (iii) *Single-seed headline runs.* The in-sample 0.685 and LOCO 0.680 are single-seed, and the proxy-vs-from-scratch separation is established at lr=10^{-5} (the from-scratch arm diverges at 2×10^{-5}, which itself demonstrates the adapted arm's stability advantage). (iv) *Retrospective drift signal.* The drift predictor's r=0.81 targets retrospective δ; the regime classifier's batch aggregation is what

converts it into a deployable signal, and its low-drift validation corpus is a bundled 20-query stable-knowledge set standing in for FreshQA (whose Wayback features require the full pipeline). The margin to τ_{regime} on other low-drift corpora is an open question.

6 Conclusion

This paper has studied when a web-backed RAG system should refresh cached evidence, on two complementary corpora separated by a measurable drift gap (Cohen's d=2.15). The best refresh policy depends on drift regime: *Always-Refresh* is the worst system on FreshQA (F1 0.173) and the best on TimeVarying (F1 0.705), so no fixed policy wins in both regimes. A proxy-trained DeBERTa router collapses zero-shot across the gap (F1 0.422) but recovers to F1 0.685 with n=60 labeled target queries, and the gain saturates at k=4; the proxy prior is what carries the few-shot improvement, since a generic-init control at the same budget plateaus near F1 0.19. A *bounded* channel decomposition attributes the low-drift anomaly to the refreshed evidence itself, with retrieval excluded by the same-URL design and the reranker and generator channels each bounded below the 0.019 gap; a *direct* $2\times2\times3$ ablation on FreshQA confirms the reranker bound at +0.005 F1 on the corpus where the anomaly occurs. A corpus-level regime classifier, averaging a stale-side drift predictor over a held-out batch of $\sim$20 queries and thresholding at τ_{regime}, separates the two regimes with no overlap and selects the policy family that matches, or comes within 0.02 of, the best fixed policy on either side. The consolidated component view is given in §4.5.

The practical takeaway is that the drift regime rather than the per-query signal is the right granularity at which to pick a refresh policy. A deploying engineer does not need a per-query router to benefit from this result; a cheap corpus-level classifier over a few dozen unlabeled queries is enough. This suggests that cost-aware refresh decisions for RAG can be made at deployment time with no target labels and only modest engineering.

Practitioner guidance.

- **Detect the regime rather than the query.** Deploy a cheap corpus-level drift predictor (a 6-feature MLP (multi-layer perceptron), or any stale-side scalar) and average it over 20−40 deployment queries. Use the resulting $\overline{\hat{\delta}}$ to pick between a low-drift policy family (Keyword Router or Never Refresh) and a high-drift family (Always Refresh or an adapted trained router). Calibrate τ_{regime} on your own corpus before deploying.
- **Use simple rules when no target labels are available.** The Keyword Router (F1 0.612 at 29% refresh on TimeVarying) is a strong default that requires no training and transfers across drift regimes without adaptation.
- **Apply a small adaptation step when a proxy-trained router is in use.** 5 epochs of MSE fine-tuning on as few as four labeled target queries suffices to recalibrate a DeBERTa-v3-small router from one drift regime to another, with the gain saturating at k=4 and holding flat through k=49. LOCO cross-validation confirms the recovery is not memorization.

- **Treat low-drift refresh as a perturbation.** When stale≈fresh, refreshing can hurt answer quality by adding evidence noise; a flat or slightly negative F1-curve under increasing refresh rate is the expected signature. The direct FreshQA ablation confirms the mechanism: the reranker channel contributes only +0.005 F1, leaving refreshed evidence itself as the operative channel.

Declarations

Data availability

The TimeVarying benchmark, the adapted DeBERTa-v3-small router checkpoint, and the full experiment logs are released with the code package at the public repository linked in the abstract. The FreshQA dev corpus is publicly available at https://github.com/lil-lab/freshqa; we use a subset of 74 queries with publicly cached Wayback snapshots. The Codex GPT-5.4 API used for answer generation is commercial; the exact prompts and response logs are included in the supplementary material.

Conflicts of interest

The authors declare no competing financial or personal interests that could have influenced the work reported in this paper.

Funding

This work received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors. The open-source language-model API credits used during development were covered by personal accounts of the authors.

Acknowledgments

We thank the anonymous reviewers for their constructive comments. We also thank the maintainers of the FreshQA, Self-RAG, and BGE-base-en-v1.5 model artifacts used in this study.

References

Asai A, Wu Z, Wang Y, et al (2024) Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In: Proceedings of ICLR

Cafarella M, Halevy A, Lee H, et al (2017) Ten years of web tables. In: Proceedings of VLDB

Devlin J, Chang MW, Lee K, et al (2019) BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of NAACL-HLT

El-Yaniv R, Wiener Y (2010) On the design and analysis of the average-case cost of selective sampling. Information Processing Letters 110(16):726–730

Geifman Y, El-Yaniv R (2017) Selective classification for deep neural networks. In: Proceedings of NeurIPS

He P, Liu X, Gao J, et al (2021) DeBERTa: Decoding-enhanced BERT with disentangled attention. In: Proceedings of ICLR

Howard J, Ruder S (2018) Universal language model fine-tuning for text classification. In: Proceedings of ACL

Jiang Z, Xu FF, Gao L, et al (2023) Active retrieval augmented generation. In: Proceedings of EMNLP

Kwiatkowski T, Palomaki J, Redfield O, et al (2019) Natural questions: A benchmark for question answering research. Transactions of the Association for Computational Linguistics

Press O, Zhang M, Min S, et al (2023) Measuring and narrowing the compositionality gap in language models. In: Findings of EMNLP, arXiv:2210.03350

Sarthi P, Abdullah S, Tuli A, et al (2024) RAPTOR: Recursive abstractive processing for tree-organized retrieval. In: Proceedings of ICLR

Thakur N, Reimers N, Rücklé A, et al (2021) BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In: Advances in Neural Information Processing Systems

Trivedi H, Balasubramanian N, Khot T, et al (2023) Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In: Proceedings of ACL, arXiv:2212.10509

Vu T, He Y, Jiang Y, et al (2024a) FreshLLMs: Refreshing large language models with search engine augmentation. In: Findings of ACL, arXiv:2310.03214

Vu T, Iyyer M, Wang X, et al (2024b) FreshLLMs: Refreshing large language models with search engine augmentation. In: Findings of ACL

Xiao S, Liu Z, Zhang P, et al (2023) C-Pack: Packaged resources to advance general chinese embedding. arXiv preprint arXiv:230907597

Yan S, Gu J, Zhu Y, et al (2024a) Corrective retrieval augmented generation. arXiv preprint arXiv:240115884

Yan SQ, Gu JC, Zhu Y, et al (2024b) Corrective retrieval augmented generation. arXiv preprint arXiv:240115884

Yao Z, Qi W, Pan L, et al (2025) SeaKR: Self-aware knowledge retrieval for adaptive retrieval augmented generation. In: Proceedings of ACL